

Object Oriented Analysis and Design

Design Patterns: Singleton and Prototype

Vladimir Soroka



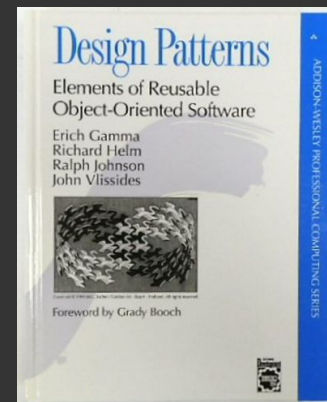
Why Design Patterns?

- Software design is hard — complexity grows with scale
- Patterns offer time-tested, reusable solutions
- They make code more understandable, flexible, and maintainable
- Help developers communicate with a shared vocabulary



The Gang of Four

- *Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides*
- Published the landmark book:
 - **“Design Patterns: Elements of Reusable Object-Oriented Software” (1994)**
- Introduced 23 classic design patterns
- Grouped into 3 categories:
 - **Creational** – object creation
 - **Structural** – class/object composition
 - **Behavioral** – object interaction & responsibility



Types of Design Patterns

Type	Examples
Creational	Singleton, Prototype, Factory Method
Structural	Adapter, Decorator, Composite
Behavioral	Observer, Strategy, Command

The Singleton Pattern

- Ensures a class has **only one instance**
- Provides a **global point of access** to it
- Often used for:
 - Config managers
 - Logging
 - Resource control (e.g., database connections)



Singleton UML

```
@startuml
skinparam classAttributeIconSize 0
' Singleton base class for clarity
class Singleton {
    - static instance
    - Singleton()
+ static getInstance()
}

class ConfigManager {
    - settings: Map<String, String>
    - ConfigManager()
+ getSetting(key: String): String
+ setSetting(key: String, value: String): void
+ getInstance(): ConfigManager
}
```

```
class Logger {
    - logLevel: String
    - Logger()
+ log(msg: String): void
+ getInstance(): Logger
}

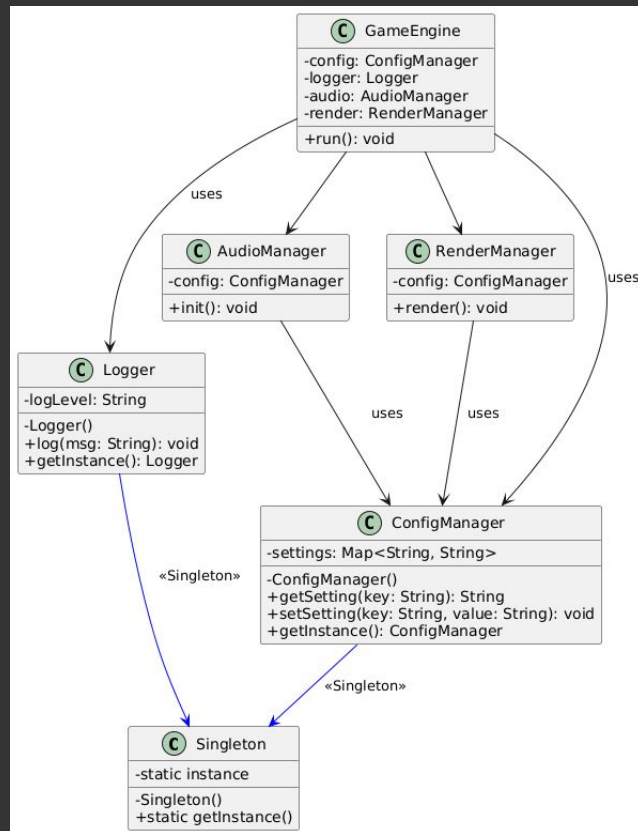
class GameEngine {
    - config: ConfigManager
    - logger: Logger
    - audio: AudioManager
    - render: RenderManager
+ run(): void
}

class AudioManager {
    - config: ConfigManager
+ init(): void
}
```

```
class RenderManager {
    - config: ConfigManager
+ render(): void
}

ConfigManager -[#blue]-> Singleton : <<Singleton>>
Logger -[#blue]-> Singleton : <<Singleton>>
GameEngine --> ConfigManager : uses
GameEngine --> Logger : uses
GameEngine --> AudioManager
GameEngine --> RenderManager
AudioManager --> ConfigManager : uses
RenderManager --> ConfigManager : uses

@enduml
```



The Singleton Code

```
#include <iostream>
#include <mutex>

class ConfigManager {
private:
    // Private constructor to prevent direct instantiation
    ConfigManager() {
        std::cout << "ConfigManager initialized.\n";
    }

    // Delete copy constructor and assignment operator
    ConfigManager(const ConfigManager&) = delete;
    ConfigManager& operator=(const ConfigManager&) = delete;

public:
    // Static method to access the single instance
    static ConfigManager& getInstance() {
        static ConfigManager instance; // Guaranteed to be thread-safe in C++11 and later
        return instance;
    }

    void setOption(const std::string& key, const std::string& value) {
        std::cout << "Setting " << key << " = " << value << "\n";
    }

    std::string getOption(const std::string& key) {
        // This is just a placeholder
        return "dummy value for " + key + " ";
    }
};
```


Singleton: An Antipattern?

- **Global State Hidden**
 - Singletons often act like global variables. They introduce hidden dependencies because any part of the code can access them without explicitly passing them around.

```
class AnalyticsTracker {  
public:  
    void trackEvent(const std::string& event) {  
        Logger::getInstance().log("Tracking event: " + event);  
    }  
};
```



Singleton: An Antipattern?

- **Singletons make unit testing difficult because:**
 - You can't easily replace the Singleton with a mock
 - The state persists across tests unless explicitly reset
 - Dependency injection becomes harder

```
// Singleton with mutable state
class UserSession {
private:
    std::string currentUser;
    UserSession() {}
public:
    static UserSession& getInstance() {
        static UserSession instance;
        return instance;
    }
    void login(const std::string& user) { currentUser = user; }
    std::string getUser() const { return currentUser; }
};
```

```
// Test A
TEST(LoginTest) {
    UserSession::getInstance().login("Alice");
    ASSERT_EQ(UserSession::getInstance().getUser(), "Alice");
}

// Test B
TEST(DefaultUserTest) {
    ASSERT_EQ(UserSession::getInstance().getUser(), ""); // FAILS
}
```

Singleton: An Antipattern?

- Thread-Safety and Lifecycle Issues
 - In multi-threaded systems, ensuring that Singleton instantiation is safe and efficient adds complexity (double-checked locking, etc.).

```
class Logger {  
    static Logger* instance;  
public:  
    static Logger* getInstance() {  
        if (!instance) {  
            instance = new Logger(); // NOT thread-safe  
        }  
        return instance;  
    }  
};
```



Singleton: An Antipattern? (cont.)

- Violates SRP (Single Responsibility Principle)
 - Singleton often mixes two responsibilities:
 - Managing its own business logic
 - Managing its own lifecycle

```
class SettingsManager {  
private:  
    static SettingsManager* instance;  
    std::map<std::string, std::string> settings;  
    SettingsManager() { loadFromFile(); }  
  
    void loadFromFile() {  
        // Reads settings from a file  
    }  
  
public:  
    static SettingsManager* getInstance();  
    std::string get(const std::string& key);  
};
```



What's the verdict?

Good Use Case	Why it works here
Logger	Needs a global access point, stateless
ConfigManager	Application-wide read-only settings
ConnectionPool	Centralized control, few instances
MainWindow, EventBus	Central hub in GUI or pub-sub systems

Or Use Dependency Injection instead

The Prototype Pattern

- **Definition:** A creational pattern that allows objects to be cloned instead of created from scratch.
 - Create new objects by copying existing ones
 - Avoids cost of complex initialization
 - Common in scenarios with many similar objects



Prototype UML

```
@startuml
class ContractTemplate
implements Document {

- clientName: string

- terms: string

- signatureBlock: string

+ clone(): Document

+ print(): void

}

class ReportTemplate implements Document {

- title: string

- body: string

- footer: string

+ clone(): Document

+ print(): void

}

class DocumentManager {

- templates: Map<string, Document>

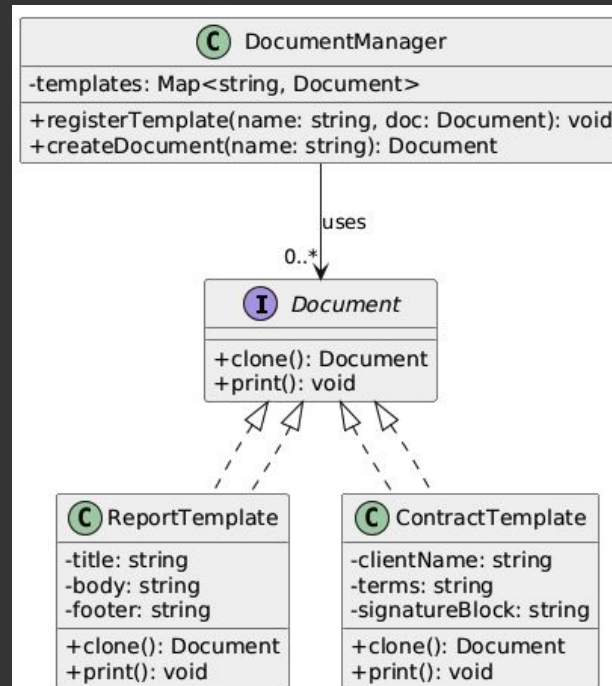
+ registerTemplate(name: string, doc: Document): void

+ createDocument(name: string): Document

}

DocumentManager --> "0..*" Document : uses
Document <|.. ReportTemplate

@enduml
```



The Prototype Code



```
#include <iostream>
#include <string>

class Document {
public:
    virtual Document* clone() const = 0;
    virtual void print() const = 0;
    virtual ~Document() {}
};

class ReportTemplate : public Document {
    std::string title;
    std::string content;
public:
    ReportTemplate(const std::string& t, const std::string& c)
        : title(t), content(c) {}

    // Copy constructor
    ReportTemplate(const ReportTemplate& other)
        : title(other.title), content(other.content) {}

    Document* clone() const override {
        return new ReportTemplate(*this); // Uses copy constructor
    }

    void print() const override {
        std::cout << "Report: " << title << "\nContent: " << content << "\n";
    }
};
```

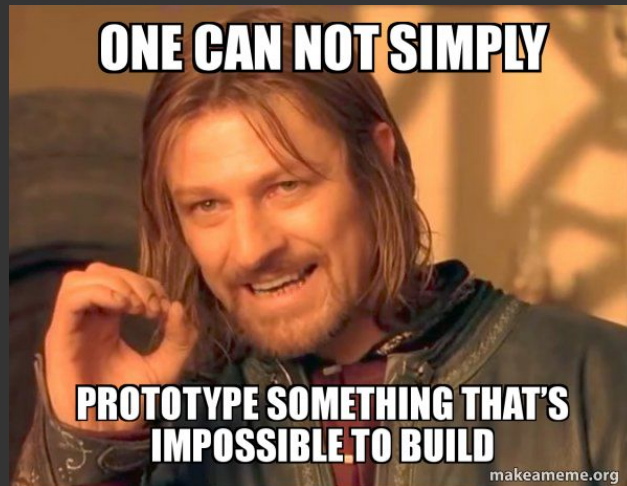

Prototype: Problems - Deep/Shallow

Problem:

- In simple cases, cloning is trivial (e.g., copy integers or strings).
- In real-world systems, objects have **pointers, collections, or nested objects** — and you must choose between **shallow** and **deep copy**.

Consequence:

- Developers forget or misjudge which type is appropriate.
- Shallow copy leads to shared state, double frees, or corruption.
- Deep copy leads to performance hits or duplication of unnecessary data.



Prototype: Problems -

Encapsulation?

Problem:

To write a clone() method, the **internal structure** of a class must be known to implement a correct copy.

Consequence:

- Violates encapsulation principle.
- Clients need to know how the object is constructed or composed to clone it correctly (especially if subclassing).

```
class Settings {  
    std::string apiKey; // private sensitive data  
public:  
    Settings(const std::string& key) : apiKey(key) {}  
  
    Settings* clone() const {  
        return new Settings(apiKey); // exposes internal field  
    }  
  
    void print() const { std::cout << "API: " << apiKey << "\n"; }  
};
```

Prototype: Problems - Consistency

Problem:

Cloned objects might **require reinitialization** or adjustment (e.g., new ID, timestamp, unique pointer, database handle).

Consequence:

- Clones may retain inappropriate state
- Risk of logic errors if clones aren't “reset”

```
class UserSession {  
    std::string username;  
    bool isLoggedIn;  
public:  
    UserSession(const std::string& user)  
        : username(user), isLoggedIn(true) {}  
  
    UserSession* clone() const {  
        return new UserSession(*this); // ❌ copies isLoggedIn = true  
    }  
  
    void print() const {  
        std::cout << username << " isLoggedIn = " << isLoggedIn << "\n";  
    }  
};
```

Prototype: Problems - Memory Management

Problem:

In languages without garbage collection (like C++), every clone must be deleted manually unless using smart pointers.

Consequence:

- High chance of memory leaks
- Added complexity for copy lifecycle tracking

```
class Email : public Document {  
public:  
    Email* clone() const override {  
        return new Email(*this); // ✅ but...  
    }  
};  
  
int main() {  
    Email* email1 = new Email();  
    Email* copy = email1->clone();  
  
    // ❌ forget to delete => memory leak  
    // delete copy; // Oops, missed it  
  
    delete email1;  
}
```